

これは何？

Google ADKを使って、ストーリー生成から画像・音声・HTMLビューアまでを一気通貫で出力する「4コマ漫画エージェント」を作って遊んだメモです。UIはシンプルですが、生成フローがはっきりしているので学習にも向いていました。

構成ざっくり

- **Agent本体**: `comic_agent/agent.py`
 - ADKの **Agent** を定義
 - 起動時の挨拶や質問の流れを日本語で固定
 - ツールの呼び出し順を `instruction` で明示
 - **ツール群**: `comic_agent/tools.py`
 - ストーリー生成 / 画像生成 / 音声生成 / HTML生成の4本柱
 - 失敗時はテンプレやモックにフォールバック
 - **スキーマ**: `comic_agent/schemas.py`
 - `ComicStory` / `Panel` / `CharacterDesign` を Pydantic で定義
-

セットアップ

```
python -m venv .venv
.venv\Scripts\activate # Windows
pip install -r requirements.txt
copy .env.example .env
```

`.env` はここだけ触る想定です。

- `GOOGLE_API_KEY` を入れる
- 画像/TTSを本物にしたいときだけ `USE_REAL_*` を `true`

```
GOOGLE_API_KEY=your_google_api_key_here
USE_REAL_STORY_GEN=true
USE_REAL_IMAGE_GEN=false
USE_REAL_TTS=false
```

ADKの導入メモ

ADK本体は `google-adk` パッケージです。今回は `pip install -r requirements.txt` で一緒に導入しています。念のため `adk --help` が通るか確認し、通らない場合は仮想環境の有効化を見直します。

実行方法

```
adk web comic_agent
# or
adk run comic_agent
```

ツールの流れ（ざっくり）

ADKのAgentはこの順番でツールを叩くように instruction に書いています。

1. `develop_story`

- Geminiで **起承転結 + 4コマJSON** を生成
- 4コマ/日本語チェックに通らなければ最大リトライ
- 失敗時はテンプレでフォールバック

2. `generate_panels`

- 1枚の画像の中に **2x2で4コマ** を描かせる
- 失敗時はPillowで簡易モック画像
- 右上→左上→右下→左下の読み順に配列を調整しているのが地味にポイント

3. `narrate_comic`

- セリフだけ抽出してTTS
- Gemini TTSのRAW PCMにWAVヘッダーを付けて保存
- セリフが空ならスキップ

4. `publish_comic`

- Jinja2で `index.html` を生成
- `comic.png` と `panel_*.wav` をHTMLに埋め込む

図解フロー（文章で図解）

```
ユーザー入力
↓
develop_story (Gemini / 4コマJSON)
↓
generate_panels (Gemini or Pillow)
↓
narrate_comic (Gemini TTS or Mock)
↓
publish_comic (Jinja2 HTML)
↓
output/index.html を開く
```

スクショ導線（撮るならここ）

- `adk web comic_agent` 起動直後の挨拶と入力例
 - `output/comic.json` の中身（4コマ構成がわかる部分）
 - `output/comic.png` の生成結果
 - `output/index.html` をブラウザで開いた画面
 - `output/` フォルダの生成物一覧
-

出力物

`output/` にまとめて出ます（実行のたびに上書き）。

- `comic.json`
 - `comic.png`
 - `panel_*.wav`
 - `index.html`
-

ちょっとした工夫メモ

- 英語が混ざると雰囲気壊れるので、**日本語チェックを厳格**にしている
 - 4コマの数ズレると破綻するので、**長さ検証とリトライ**は重要
 - 画像生成は **1枚で4コマ** にしてUIを単純化した
-

参照したもの（自分メモ）

- `README.md`（セットアップ/実行/出力）
 - `.env.example`（環境変数の一覧）
 - `requirements.txt`（依存ライブラリ）
 - `comic_agent/agent.py`（Agent定義と対話フロー）
 - `comic_agent/tools.py`（生成処理の本体）
 - `comic_agent/schemas.py`（Pydantic定義）
 - `comic_agent/templates/viewer.html`（HTMLビューア）
 - `verify_pipeline.py`（モック動作検証）
 - `debug_tts.py`（TTSレスポンスの形式確認）
-

おわりに

「生成の一連フローをADKで書き切る」練習としてちょうど良かったです。次は画像のレイアウト固定や、セリフの演出（吹き出し配置）をもう少し詰めたい。